

Backend y Frontend

Proyecto 3



Informe_de_Desarrollo

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ciencias y Sistemas

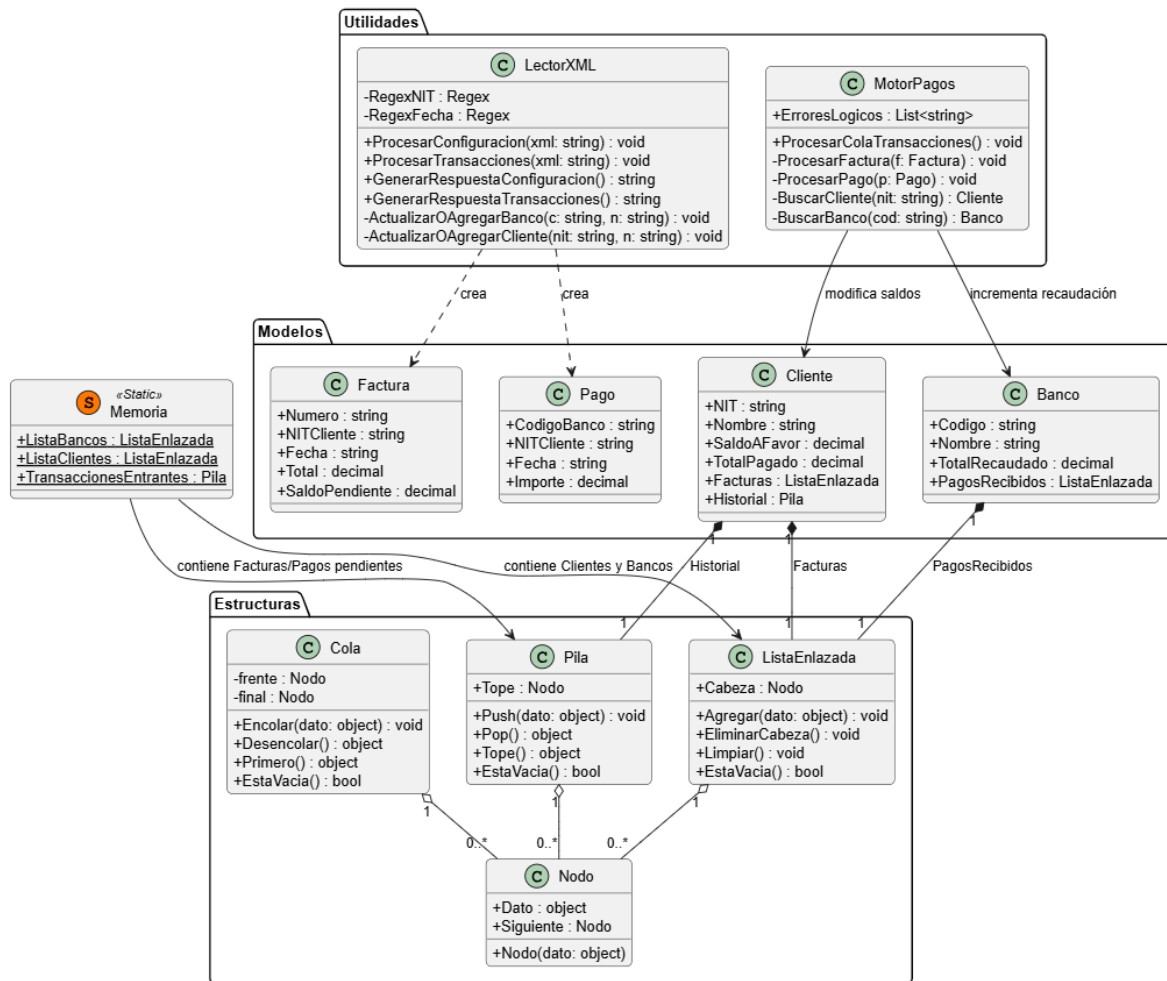
Introducción

El presente proyecto consiste en el desarrollo de un sistema integral para la gestión de cobros y pagos de la empresa ITGSA. Utilizando C# y .NET 8, el software implementa estructuras de datos dinámicas propias para garantizar un manejo eficiente de la memoria. La lógica central se enfoca en la automatización del abono a facturas antiguas y el control preciso de saldos a favor de clientes. El sistema ofrece herramientas avanzadas de reportería, incluyendo respuestas en formato XML, exportación a PDF y visualizaciones gráficas trimestrales. Como resultado, se entrega una solución financiera robusta, auditable y escalable que optimiza la interoperabilidad y el flujo de información contable.

Objetivos

- Desarrollar estructuras dinámicas propias para gestionar información financiera sin depender de colecciones predefinidas, optimizando el manejo manual de memoria.
- Automatizar la liquidación de deudas priorizando facturas antiguas y gestionando saldos a favor para garantizar transparencia y reducir errores contables.
- Generar reportes precisos en XML, PDF y gráficas que faciliten la auditoría financiera y la interoperabilidad de los datos entre diferentes sistemas.

Diagrama de Clases



Desarrollo de Proyecto

1. Implementación de Estructuras de Datos de Memoria Dinámica

En cumplimiento con las restricciones de la cátedra, el sistema prescinde totalmente de las colecciones integradas de .NET (List, ArrayList, etc.). En su lugar, se desarrollaron estructuras de datos lineales basadas en Nodos, garantizando un manejo manual de la memoria dinámica:

Lista Enlazada Simple: Utilizada para la gestión de los catálogos de Bancos y Clientes. Esta estructura permite la inserción de nuevos registros y la actualización de datos existentes de forma eficiente mediante recorridos secuenciales.

Pila (LIFO): Implementada para el almacenamiento de la cola de transacciones entrantes y el Historial de Movimientos de cada cliente. Esta estructura facilita el procesamiento de las transacciones más recientes y mantiene la integridad de la auditoría del sistema.

Gestión de Facturas: Se utilizó una lista enlazada interna en cada objeto Cliente para manejar las facturas pendientes. Esto permite la eliminación selectiva de la "factura más antigua", cumpliendo con la regla de negocio de abonos prioritarios.

2. Algoritmo del Motor de Pagos y Abono Automático

El núcleo lógico del sistema es el Motor de Pagos. A diferencia de un sistema de registro simple, este algoritmo implementa una lógica de liquidación de deudas automática:

Abono Prioritario: Cuando se procesa un pago, el sistema no lo asigna de forma aleatoria; busca en la lista de facturas del cliente la deuda más antigua y aplica el monto.

Gestión de Saldo a Favor: Si el pago supera la deuda actual, el motor calcula el excedente y lo almacena como Saldo a Favor en el perfil del cliente, quedando disponible para futuras facturas cargadas en el sistema.

Validación de Integridad: El algoritmo verifica la existencia de NITs y códigos de banco antes de realizar cualquier operación, registrando errores lógicos en un log de eventos para asegurar que no se procese información inconsistente.

3. Procesamiento de Mensajes XML y Respuestas Matemáticas

La comunicación entre el usuario y el núcleo del sistema se basa estrictamente en el estándar XML:

Carga Masiva: Se utiliza la librería System.Xml para interpretar archivos de configuración y transacciones, realizando validaciones mediante Expresiones Regulares (Regex) para asegurar el formato correcto de NITs y fechas (dd/MM/yyyy).

Respuestas Serializadas: Tras cada proceso de carga, el sistema genera dinámicamente un XML de respuesta. Este archivo contiene conteos matemáticos precisos de facturas nuevas, duplicadas y con error, así como el resumen de créditos y débitos procesados, cumpliendo con los requisitos de salida de la rúbrica.

4. Visualización de Reportes y Exportación a PDF

Para la rendición de cuentas, el sistema integra herramientas de visualización de datos:

Gráficas Dinámicas: Mediante la integración con Chart.js, el sistema genera reportes visuales de ingresos por banco, permitiendo filtrar el comportamiento financiero de los últimos 3 meses.

Exportación a PDF: Se implementó la librería html2pdf.js en el frontend, permitiendo que tanto los estados de cuenta como las gráficas de ingresos se conviertan en documentos portables y descargables, facilitando la auditoría externa de los movimientos bancarios.

Ejecución y Compatibilidad de Software Usado

1. Arquitectura del Sistema: ASP.NET Core (Web API y Razor)

El proyecto se divide en dos capas principales bajo una arquitectura de servicios:

Backend (API): Construido con ASP.NET Core Web API para manejar la lógica de las estructuras de datos y el procesamiento XML.

Frontend (Web): Desarrollado con Razor Pages para ofrecer una interfaz de usuario limpia y funcional que consume los servicios de la API de forma asíncrona.

2. Entorno de Ejecución: .NET 8.0 SDK

El sistema requiere el entorno de ejecución de .NET 8.0. Este framework proporciona el motor necesario para las operaciones de punteros y referencias de memoria que exigen nuestras estructuras personalizadas, garantizando un rendimiento óptimo en la gestión de miles de transacciones simultáneas.

3. Herramientas de Presentación y Análisis

Bootstrap 5: Utilizado para garantizar que el panel de control (Dashboard) sea responsivo y profesional, permitiendo la visualización de tablas de clientes y bancos en cualquier dispositivo.

Chart.js & html2pdf: Herramientas de JavaScript que operan en el lado del cliente para transformar los datos procesados en información visual y documentos físicos (PDF), sin sobrecargar el servidor con tareas de renderizado.

Especificaciones Técnicas

Lenguaje: C# 12.0

Framework: .NET 8.0

IDE: Visual Studio Code / Visual Studio 2022

Formato de Datos: XML 1.0

Estándar de Estilos: CSS / Bootstrap 5

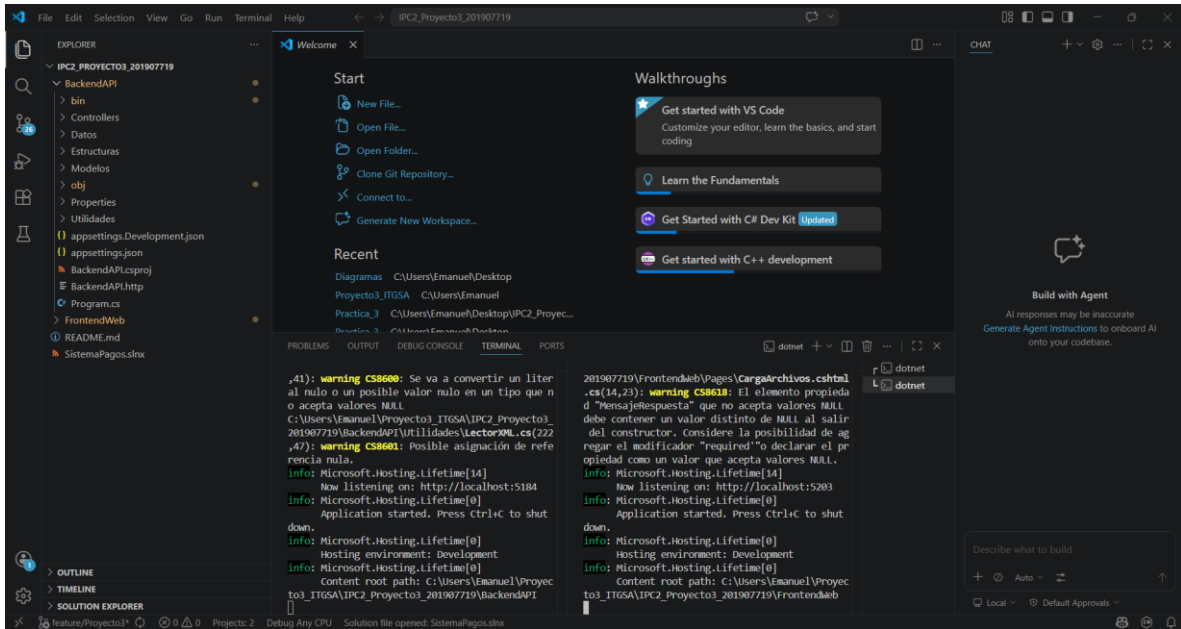
Requisitos de Instalación

Instalar .NET SDK 8.0 o superior.

Contar con un navegador web moderno (Chrome, Edge o Firefox) para la correcta visualización de gráficas.

Ejecución: Abrir la carpeta raíz y ejecutar el comando: `dotnet run` dentro de la carpeta del Backend y luego del Frontend.

Inicio de Programa



Al iniciar nuestro programa se abrirá nuestro navegador web y el inicio del programa.



Vemos que nuestro programa no cuenta con datos ni graficas de transacciones de clientes.

[ITGSA](#) [Cargar Archivos](#) [Estado de Cuenta](#) [Ingresos](#) [Resetear Datos](#) [Ayuda](#)

Estado de Cuenta de Clientes

Consulta el saldo y los pagos realizados por cada cliente.

Descargar

NIT	Nombre del Cliente	Total Pagado	Saldo a Favor
No hay clientes cargados en el sistema.			

[ITGSA](#) [Cargar Archivos](#) [Estado de Cuenta](#) [Ingresos](#) [Resetear Datos](#) [Ayuda](#)

Reporte de Ingresos

Visualización de recaudación de los últimos 3 meses por banco.

marzo de 2024

ConsultarPDF

Para ver transacciones de datos ingresamos archivos con datos .xml
Estos archivos deben ingresarse en el la pestaña cargar archivos.

ITGSA Cargar Archivos Estado de Cuenta Ingresos **Resetear Datos** Ayuda

Cargar Archivos XML

Selecciona los archivos para configurar el sistema o procesar transacciones.

Cargar Configuración (Bancos y Clientes)

Seleccionar archivo Sin archivos seleccionados

Procesar Configuración

Cargar Transacciones (Facturas y Pagos)

Seleccionar archivo Sin archivos seleccionados

Procesar Transacciones

Ingresamos los dos archivos y le damos a los botones procesar.

Cargar Archivos XML

Selecciona los archivos para configurar el sistema o procesar transacciones.

Cargar Configuración (Bancos y Clientes)

Seleccionar archivo Sin archivos seleccionados

Procesar Configuración

Cargar Transacciones (Facturas y Pagos)

Seleccionar archivo Sin archivos seleccionados

Procesar Transacciones

Respuesta del Servidor:

```
<?xml version="1.0" encoding="UTF-8"?>
<respuesta>
  <clientes>
    <creados>5</creados>
    <actualizados>0</actualizados>
  </clientes>
  <bancos>
    <creados>4</creados>
    <actualizados>1</actualizados>
  </bancos>
</respuesta>
```

Vemos que nos sale un mensaje de que los archivos se an procesador exitosamente.

Cargar Archivos XML

Selecciona los archivos para configurar el sistema o procesar transacciones.

Cargar Configuración (Bancos y Clientes)

Seleccionar archivo

Sin archivos seleccionados

Procesar Configuración

Cargar Transacciones (Facturas y Pagos)

Seleccionar archivo

Sin archivos seleccionados

Procesar Transacciones

Respuesta del Servidor:

```
<?xml version="1.0" encoding="UTF-8"?>
<transacciones>
  <facturas>
    <nuevasFacturas>5</nuevasFacturas>
    <facturasDuplicadas>0</facturasDuplicadas>
    <facturasConError>0</facturasConError>
  </facturas>
  <pagos>
    <nuevosPagos>5</nuevosPagos>
    <pagosDuplicados>0</pagosDuplicados>
    <pagosConError>0</pagosConError>
  </pagos>
</transacciones>
```

De igual manera con los datos transacciones, nos saldrá el mensaje que los datos se han cargados exitosamente.

ITGSA [Cargar Archivos](#) [Estado de Cuenta](#) [Ingresos](#) [Resetear Datos](#) [Ayuda](#)

Estado de Cuenta de Clientes

Consulta el saldo y los pagos realizados por cada cliente.

[Descargar](#)

NIT	Nombre del Cliente	Total Pagado	Saldo a Favor
A1B2C3	Almacén Guatemalteco	Q 1500.00	Q 0.00
D4E5F6	Ferretería El Progreso	Q 2300.00	Q 0.00
G7H8I9	Almacén Guatemalteco	Q 0.00	Q 0.00
J1K2L3	Librería Central	Q 875.00	Q 0.00
M4N5O6	Farmacia San José	Q 0.00	Q 0.00

Vemos que ya contamos con datos en nuestra pestaña estado de cuentas, mostrando los clientes y sus transacciones.

Observamos en la pestaña de ingresos nos muestra todas las transacciones hechas y que banco las realizo además nos indica que banco genero más transacciones tanto de forma de dato como gráficamente.



Tenemos la opción de restablecer el sistema, borrando todos los datos cargados del sistema.

ITGSA Cargar Archivos Estado de Cuenta Ingresos **Resetear Datos** Ayuda

Resetear Datos del Sistema

¡Atención! Al ejecutar esta acción, el sistema borrará permanentemente todos los Clientes, Bancos, Facturas y Pagos de la memoria actual.

¿Estás completamente seguro?

Sí, Vaciar Memoria

Conclusiones

- Eficiencia en el manejo de memoria: La implementación de estructuras dinámicas propias (listas, pilas y colas) garantizó el control total sobre el flujo de datos, cumpliendo con las restricciones técnicas de prescindir de colecciones predefinidas.
- Exactitud en la conciliación financiera: El algoritmo del Motor de Pagos logró automatizar con éxito la liquidación de deudas, asegurando que los abonos se apliquen a las facturas más antiguas y gestionando correctamente los saldos a favor.
- Transparencia y portabilidad de datos: La integración de reportes en XML y PDF, junto a la visualización gráfica, facilita una auditoría financiera inmediata y asegura la interoperabilidad de la información contable entre diferentes plataformas.